Fortnite Performance Dashboard

System Design — ASP.NET Core MVC with AI-Assisted Coaching for Fortnite

Group 15

Note: This document covers the System Design phase only — translating the approved requirements into a concrete application architecture: layers, modules, data flow, integration with FortniteAPI.io, and the design decisions/trade-offs behind each choice, ready for Database Design and Sprint Planning.

Business Problem → Requirement Analysis → System Design → Database Design → Sprint Planning → Development

1. What is System Design?

The process of deciding how a software system will be structured to meet its requirements — which architectural style, layers, components, and integrations to use, how they communicate, and why — before any database schema or code is written.

2. Design Goals

- Keep controllers thin and push business logic into a dedicated service layer, so the codebase stays testable and maintainable for a small semester-length team.
- Isolate the FortniteAPI.io dependency behind a single client/service so the free-tier rate limit (10 requests/min) is respected and the rest of the app never talks to the API directly.
- Make the rule-based coaching engine swappable for a future ML/LLM engine without touching the database or controllers (per the Future Enhancements scope).
- Support two roles (Player, Administrator) with clearly separated permissions.
- Keep the architecture web-responsive only — no native mobile app — per the stated constraints.

3. Architectural Style

The system follows a layered (N-tier) architecture on top of the ASP.NET Core MVC pattern. MVC separates how a request is received and rendered (Controller/View) from what the application does with data (Model), and an additional Service Layer is introduced between Controllers and EF Core so that business logic — API calls, calculations, recommendation generation — never lives inside a controller.

Layer	Responsibility
Presentation	Razor views, Bootstrap layout, Chart.js visualizations. Renders what the Controller passes it — no business logic.
Controller	Receives HTTP requests, validates input/model state, calls the Service Layer, returns a View or JSON result. Stays thin by design.
Service Layer	Owns business logic: calls the FortniteAPI.io client, calculates K/D ratio and win rate, runs the rule-based recommendation engine. The only layer allowed to call EF Core.
Data Access (EF Core)	Maps C# entities to SQL Server tables, tracks changes, executes queries via LINQ.
Database	Microsoft SQL Server — persists Users, Players, Stats, Recommendations.

Table 3.1 — Layer responsibilities.

4. High-Level Architecture Flow

Presentation (Razor / Bootstrap / Chart.js) → Controller → Service Layer (FortniteAPI.io Client + Stats Calculator + Recommendation Engine) → EF Core → SQL Server

Requests flow top-down and responses flow back up the same path. Controllers never call EF Core directly, and the Presentation layer never calls the Service Layer directly — every request is mediated by a Controller, keeping each layer replaceable in isolation.

5. Component / Module Breakdown

Component	Design Notes
Authentication & Roles	ASP.NET Core Identity-style login/registration; Role claim ("Player" / "Administrator") drives [Authorize] attributes on controllers.
Fortnite API Integration	A single FortniteApiClient service wraps all FortniteAPI.io calls, centralizing the API key, base URL, and rate-limit handling (e.g., simple in-memory throttling / retry-after).
Stats Calculation	A StatsService takes raw API data and computes K/D ratio, win rate, and accuracy trend deltas, then persists the result via EF Core.
Dashboard	Controller assembles a DashboardViewModel (current stats + recent recommendations) for Razor views; Chart.js renders trend charts client-side from JSON passed in the view.
AI-Assisted Coaching	A RecommendationEngine service applies fixed if/else or threshold rules against Stats (e.g., accuracy below X% → suggest aim training) and writes results to Recommendations.
Admin Panel	Separate AdminController restricted to the Administrator role; manages players and game-mode/category reference data, and views platform-wide aggregated stats.

Table 5.1 — Component responsibilities.

6. Sequence: Stat Sync & Coaching

1. Player clicks "Sync Stats" on the dashboard. DashboardController calls StatsService.SyncAsync(playerId).
2. StatsService calls FortniteApiClient, which requests the player's data from FortniteAPI.io.
3. StatsService validates the response, computes K/D ratio and win rate, and upserts the Stats row via EF Core.
4. StatsService calls RecommendationEngine.Generate(stats), which inserts a new Recommendations row.
5. Controller reloads the updated ViewModel and returns the Dashboard view with fresh charts and tips.

7. Design Patterns Used

Pattern	Where / Why
MVC	Overall request/response structure, built into ASP.NET Core.
Service Layer	Encapsulates business rules and third-party API calls outside controllers, per the requirement that "the service layer calls the FortniteAPI.io, handles calculations and recommendations."
Repository (via EF Core DbContext)	DbContext + DbSet already provide repository/unit-of-work semantics, so a hand-rolled repository layer is intentionally skipped to avoid needless abstraction for a small project.

Pattern	Where / Why
Strategy (for coaching rules)	RecommendationEngine exposes one interface; the current rule-based implementation can be swapped for an ML/LLM-backed implementation later without changing callers.
Dependency Injection	Built-in ASP.NET Core DI registers FortniteApiClient, StatsService, and RecommendationEngine as scoped services, keeping components testable and loosely coupled.

Table 7.1 — Applied patterns and rationale.

8. External Integration Design — FortniteAPI.io

- All calls go through one FortniteApiClient wrapper — no controller or view ever references the API directly.
- Requests are keyed by the Player's linked FortniteUsername, matching the Players table design.
- The free-tier limit of 10 requests/min is respected by disabling the "Sync" action client-side for a cooldown window after each successful sync, and returning a friendly error if the API responds with a rate-limit status.
- API failures (player not found, private stats, API downtime) are caught in the Service Layer and surfaced to the Controller as a typed result, not an unhandled exception, so the Dashboard can show a clear message instead of crashing.

9. Security Design

- Passwords are hashed (never stored in plain text) and role claims are issued at login.
- [Authorize] and [Authorize(Roles = "Administrator")] attributes gate controller actions by role.
- All Fortnite/API secrets (API key) are kept in configuration (appsettings / environment variables / secret manager), never hard-coded or committed.
- Input validation happens both client-side (basic UX) and server-side (ModelState, data annotations) since client-side checks alone are not trustworthy.
- EF Core parameterizes all queries by default, mitigating SQL injection risk.

10. Non-Functional Design Considerations

Concern	How the design addresses it
Response time	Chart.js rendering happens client-side from pre-fetched JSON; heavy computation (stat calculation) happens once per sync, not per page view.
Availability	Sync failures degrade gracefully — the dashboard still renders the last known Stats/Recommendations if FortniteAPI.io is temporarily unavailable.
Scalability	Stateless Controllers/Services allow horizontal scaling behind a load balancer if needed; SQL Server can scale vertically for this project's expected load.
Reliability	Service Layer wraps external calls in try/catch with typed results; EF Core transactions ensure Stats updates and Recommendation inserts either both succeed or neither does.

Table 10.1 — Non-functional requirements mapped to design decisions.

11. Technology Stack Summary

Layer	Technology
Backend	ASP.NET Core MVC (.NET 8), C#
Data Access	Entity Framework Core
Database	Microsoft SQL Server
Frontend	Razor Views, HTML, CSS, Bootstrap, JavaScript, Chart.js
External API	FortniteAPI.io (third-party Fortnite stats provider)
Hosting (minimum)	Server: 4GB RAM, dual-core · Client: any device with a modern browser

Table 11.1 — Confirmed technology choices from Requirement Analysis, carried into System Design.

12. Constraints Carried Into This Design

- Small semester-length team → architecture favors simplicity (Service Layer over full Repository/CQRS) rather than enterprise-scale patterns.
- Coaching is rule-based only in this phase; the Strategy-pattern RecommendationEngine interface is what keeps a future AI/LLM swap low-risk.
- Single SQL Server database; no distributed data store needed at this scale.
- Web-responsive only — the architecture assumes browser clients, not a native mobile app.

13. What Comes Next?

Requirement Analysis → System Design (this document) → Database Design → Sprint Planning → Development

Fortnite Performance Dashboard — System Design — Group 15